

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	M. SAHASRA	Reg. No.:	192425317
Section / Batch:	2024-2028	Date:	15-07-2026

Code of Conduct

I M. SAHASRA 192425317 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Sahasra

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1)

1. Aim:

To develop a Python program using Regular Expressions to automatically extract candidate information such as name, email, phone number, skills, and experience from a resume and identify eligible candidates.

2. Problem Understanding

Example:

The recruitment company receives resumes in text format. Manually checking every resume is time-consuming. Regular Expressions help automatically extract required details and shortlist candidates based on eligibility criteria.

3. Concepts Used

- Python re module
- re.search()
- re.findall()
- Regular Expression Patterns
- Conditional Statements
- Lists
- String Processing

4. Algorithm / Steps

1. Import the re module.
2. Store resume data.
3. Extract candidate name.
4. Extract email address.
5. Extract mobile number.
6. Search for technical skills.
7. Extract years of experience.
8. Check eligibility.
9. Display candidate profile.
10. Print eligibility status.

5. Python Code



```
import re
```

```
resume = """
Name: Sahasra Medarametla
Email: sahasra@gmail.com
Mobile: 9876543210
Skills: Python, SQL, Machine Learning, NLP
Experience: 3 years
"""

# Name
name = re.search(r"Name:\s*(.*)", resume)

# Email
email = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', resume)

# Mobile
mobile = re.findall(r'\b[6-9]\d{9}\b', resume)

# Skills
skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills_found = []

for skill in skills_list:
    if re.search(skill, resume, re.IGNORECASE):
        skills_found.append(skill)

# Experience
exp = re.search(r'(\d+)\s+years', resume)

experience = int(exp.group(1)) if exp else 0

print("----- Candidate Profile -----")
print("Name :", name.group(1))
print("Email :", email)
print("Mobile :", mobile)
print("Skills :", skills_found)
print("Experience :", experience, "years")

print("\nEligibility Status")

if experience >= 2 and "Python" in skills_found:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")
```

6. Sample Output

```
... ----- Candidate Profile -----
Name : Sahasra Medarametla
Email : ['sahasra@gmail.com']
Mobile : ['9876543210']
Skills : ['Python', 'SQL', 'Machine Learning', 'NLP']
Experience : 3 years

Eligibility Status
Eligible for Shortlisting
```

7. Result:

The program successfully extracted all required candidate information using Regular Expressions and correctly identified eligible candidates based on the specified criteria.

2)

1. Aim:

To develop a product search system using Regular Expressions for efficient product searching.

2. Problem Understanding:

An e-commerce website needs flexible product searching using regex.

3. Concepts Used:

- re.match()
- re.search()
- re.fullmatch()
- IGNORECASE
- Lists

4. Algorithm:

1. Start the program.
2. Import the re (Regular Expression) module.
3. Create a list containing the names of available products.
4. Accept the search keyword from the user.
5. Perform an exact match search using re.fullmatch().
6. Perform a prefix match search using re.match().
7. Perform a suffix match search using re.search() with the \$ symbol.

8. Perform a partial keyword search using `re.search()`.
9. Perform a case-insensitive search using the `re.IGNORECASE` flag.
10. Display all matching product names for each search type.
11. Count the total number of matches obtained in each search.
12. Display a summary report showing the number of exact, prefix, suffix, and partial matches.
13. Stop the program.

5. Code:

```
import re

products = [
    "Laptop",
    "Laptop Bag",
    "Wireless Mouse",
    "Gaming Keyboard",
    "USB Cable",
    "Bluetooth Speaker",
    "Smart Watch",
    "Mouse Pad"
]

keyword = input("Enter search keyword: ")

print("\nExact Match")
exact = [p for p in products if re.fullmatch(keyword, p, re.IGNORECASE)]
print(exact)

print("\nPrefix Match")
prefix = [p for p in products if re.match(keyword, p, re.IGNORECASE)]
print(prefix)

print("\nSuffix Match")
suffix = [p for p in products if re.search(keyword + r'$' , p, re.IGNORECASE)]
print(suffix)

print("\nPartial Match")
partial = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(partial)

print("\nCase Insensitive Match")
case = [p for p in products if re.search(keyword, p, re.IGNORECASE)]
print(case)

print("\n----- Report -----")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
print("Suffix Matches :", len(suffix))
print("Partial Matches :", len(partial))
```

6. Sample Input:

... Enter search keyword: Mouse

7. Sample Output:

```
Exact Match
[]

Prefix Match
['Mouse Pad']

Suffix Match
['Wireless Mouse']

Partial Match
['Wireless Mouse', 'Mouse Pad']

Case Insensitive Match
['Wireless Mouse', 'Mouse Pad']

----- Report -----
Exact Matches : 0
Prefix Matches : 1
Suffix Matches : 1
Partial Matches : 2
```

8. Result:

The program successfully searched products using exact, prefix, suffix, partial, and case-insensitive matching.

3)

1. Aim:

To validate student registration details using Regular Expressions.

2. Problem Understanding:

Validation is important before registration.

3. Concepts Used:

- re.match()
- Regex patterns
- Validation

- Conditional Statements

4. Algorithm:

1. Start the program.
2. Import the re (Regular Expression) module.
3. Accept the student's register number as input.
4. Accept the institutional email address as input.
5. Accept the course code as input.
6. Accept the semester number as input.
7. Accept the student's mobile number as input.
8. Define suitable regular expression patterns for validating each input field.
9. Validate the register number using re.match() and display whether it is valid or invalid.
10. Validate the institutional email address and display the validation result.
11. Validate the course code and display the validation result.
12. Validate the semester number and display the validation result.
13. Validate the mobile number and display the validation result.
14. Check whether all the entered details satisfy the required validation rules.
15. If all fields are valid, display "Registration Successful"; otherwise, display "Registration Failed".
16. Stop the program.

5. Code:

```
import re

reg = input("Enter Register Number: ")
email = input("Enter College Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

reg_pattern = r'^\d{2}[A-Z]{3}\d{4}$'
email_pattern = r'^[a-zA-Z0-9._%+-]+@saveetha\.com$'
course_pattern = r'^[A-Z]{3}\d{3}$'
semester_pattern = r'^[1-8]$'
mobile_pattern = r'^[6-9]\d{9}$'

status = True

if re.match(reg_pattern, reg):
    print("Register Number : Valid")
else:
    print("Register Number : Invalid")
    status = False

if re.match(email_pattern, email):
    print("Email : Valid")
else:
    print("Email : Invalid")
    status = False

if re.match(course_pattern, course):
```



```
if re.match(course_pattern, course):
    print("Course Code : Valid")
else:
    print("Course Code : Invalid")
    status = False

if re.match(semester_pattern, semester):
    print("Semester : Valid")
else:
    print("Semester : Invalid")
    status = False

if re.match(mobile_pattern, mobile):
    print("Mobile Number : Valid")
else:
    print("Mobile Number : Invalid")
    status = False

print("\nRegistration Status")

if status:
    print("Registration Successful")
else:
    print("Registration Failed")
```

6. Sample Input:

```
... Enter Register Number: 24AIM1234
Enter College Email: student@saveetha.com
Enter Course Code: NLP301
Enter Semester: 5
Enter Mobile Number: 9876543210
```

7. Sample Output:

```
Register Number : Valid
Email : Valid
Course Code : Valid
Semester : Valid
Mobile Number : Valid

Registration Status
Registration Successful
```

8. Result:

The program successfully validated all registration details and displayed the final registration status.